



Form 3

Design

**Title : Integrated Operational Management System for C3MR:
Web Admin Portal & Telegram Mini App Ecosystem**

GROUP MEMBER:

No.	Student Name	Student ID	Study Program
1.	Auza Syamil Nabawi	001202300150	Informatics
2.	Rashad Abdul Faqih	001202300149	Informatics
3.	Atthariqul Hazam Albanna	012202300122	Information System

Advisor : Dr. Adhi Setyo Santoso, ST.MBA.

**Submitted for
Capstone Design Project
Faculty of Computer Science
President University
2026**

TABLE OF CONTENT

STATEMENT OF ORIGINALITY	3
PART 3 : DESIGN	4
A. SYSTEM DESIGN	4
Overview	4
System Architecture	4
Mock-up Designs	6
B. HIERARCHICAL / ITERATIVE DESIGN	10
1. Data Flow Diagram	10
2. Interface Information Between Components	11
3. Software Engineering Design Steps	11
4. Component References and Libraries	12
5. UML Diagrams	13
C. STANDARDS USED	15
D. IMPLEMENTATION AND TESTING SCENARIO	16
A. Implementation Overview	16
B. Testing Scenarios	17
REFERENCES	20

STATEMENT OF ORIGINALITY

In my capacity as an active student at President University and as the author of the Capstone Design Project stated below:

Name : 1. Auza Syamil Nabawi – 001202300150
2. Rashad Abdul Faqih – 001202300149
3. Atthariqul Hazam Albanna – 012202300122

Faculty : Computer Science

I hereby declare that my Capstone Design Project entitled “**Integrated Operational Management System for C3MR: Web Admin Portal & Telegram Mini App Ecosystem**” is to the best of my knowledge and belief, an original piece of work based on sound academic principles. If there is any plagiarism detected in this final project, I am willing to be personally responsible for the consequences of these acts of plagiarism and will accept the sanctions against these acts in accordance with the rules and policies of President University.

I also declare that this work, either in whole or in part, has not been submitted to another university to obtain a degree.

Cikarang, 2026

Signer 1	Signer 2	Signer 3
		
Auza Syamil Nabawi 001202300150	Rashad Abdul Faqih 001202300149	Atthariqul Hazam Albanna 012202300122

PART 3 : DESIGN

Consists of:

A. SYSTEM DESIGN

B. HIERARCHICAL / ITERATIVE DESIGN

C. STANDARDS USED

D. FUNCTIONAL TESTING

A. SYSTEM DESIGN

Overview

C3MR (Collection Case & Customer Management Report) is a web-based field collection management system designed to streamline the workflow between field officers and managers in the debt collection process. The system follows a three-tier architecture consisting of a Presentation Layer, Business Logic Layer, and Data Storage Layer.

System Architecture

The system architecture is illustrated in the diagram below:

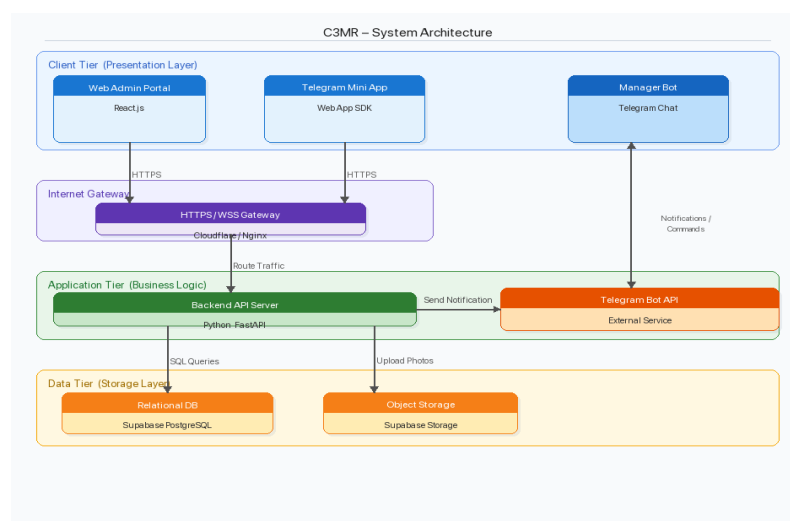


Figure A.1 C3MR System Architecture Diagram

The architecture consists of the following tiers:

1. Client Tier (Presentation Layer)

- Web Admin Portal (React.js): A browser-based dashboard used by managers to upload collection targets (CSV), assign officers, and monitor progress in real time.
- Telegram Mini App (Web App SDK): A lightweight mobile interface embedded within Telegram, used by field officers to view assigned tasks, fill visit reports, upload photo evidence, and submit results.
- Manager Bot (Telegram Chat): A Telegram bot that allows managers to receive notifications and issue commands (e.g., reassign targets, view summaries).

2. Internet Gateway

- HTTPS / WSS Gateway (Cloudflare / Nginx): Acts as a reverse proxy and load balancer handling SSL termination, rate limiting, and request routing.

3. Application Tier (Business Logic)

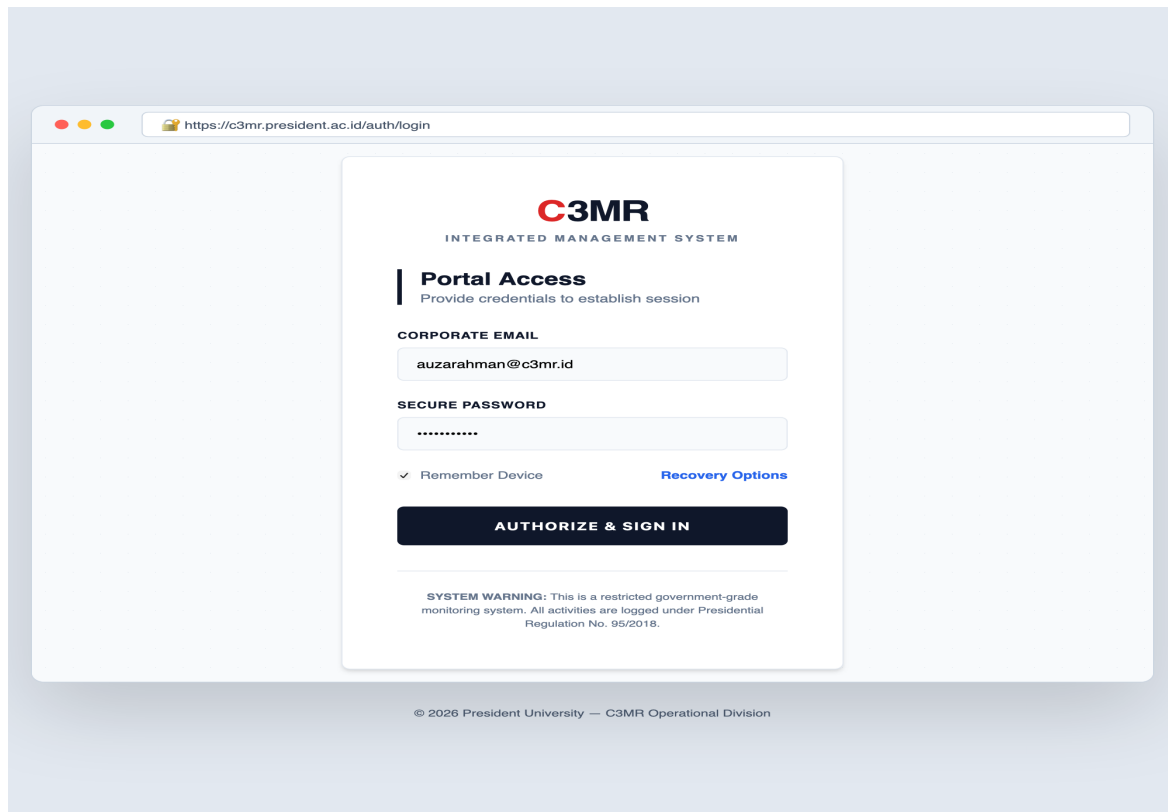
- Backend API Server (Python FastAPI): Handles all API requests, validates input data via Pydantic, executes business logic (task assignment, report processing, status updates), and communicates with the database.

4. Data Tier (Storage Layer)

- Relational Database (Supabase PostgreSQL): Stores user accounts, collection targets, visit reports, officer assignments, and audit logs.
- Object Storage (Supabase Storage): Stores photo evidence from field visits.

Mock-up Designs

Web Admin Portal – Login Page



The image shows a web browser window displaying the login page of the C3MR Integrated Management System. The browser's address bar shows the URL `https://c3mr.president.ac.id/auth/login`. The login form is centered and contains the following elements:

- C3MR** logo and the text "INTEGRATED MANAGEMENT SYSTEM".
- Portal Access** header with the instruction "Provide credentials to establish session".
- CORPORATE EMAIL** label above an input field containing `auzarahman@c3mr.id`.
- SECURE PASSWORD** label above a password input field with masked characters.
- A checkbox labeled "Remember Device" and a link for "Recovery Options".
- A dark blue button labeled **AUTHORIZE & SIGN IN**.
- A **SYSTEM WARNING** at the bottom: "This is a restricted government-grade monitoring system. All activities are logged under Presidential Regulation No. 95/2018."

At the bottom of the page, a copyright notice reads: "© 2026 President University — C3MR Operational Division".

Figure A.2a Web Admin Portal – Login Page

The Login Page is the entry point for authorized managers to access the C3MR Web Admin Portal. It includes the following interface elements:

- Email and Password input fields with validation.
- Login button that authenticates via Supabase Auth and stores a JWT token.
- Forgot Password link for credential recovery.
- Access restriction notice — only whitelisted manager accounts can log in.

Web Admin Portal – Target Management Dashboard

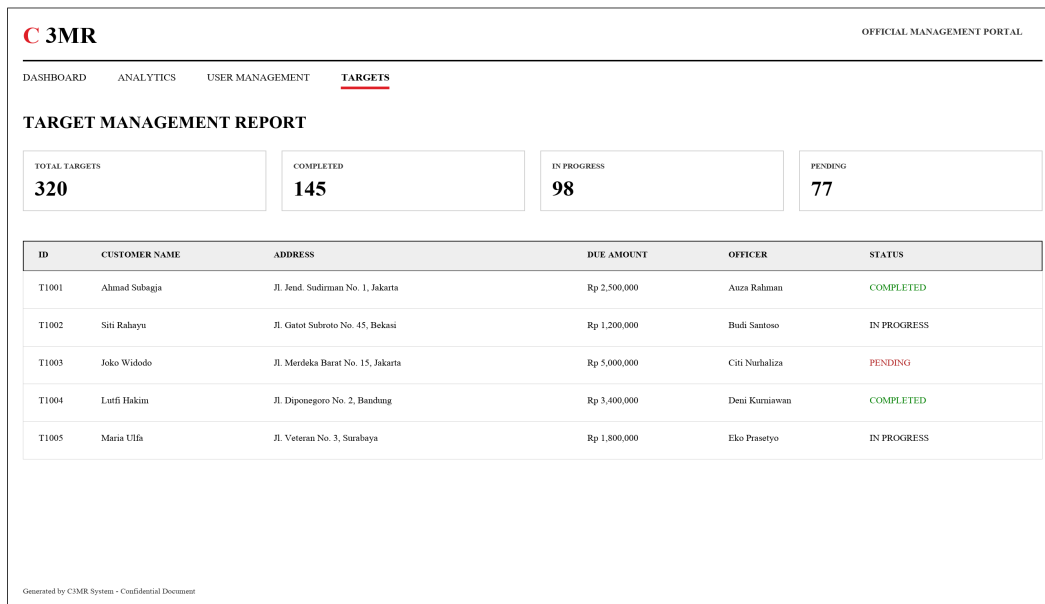


Figure A.2b Web Admin Portal – Target Management Dashboard

The Target Management Dashboard provides the following interface elements:

- Navigation Tabs: Dashboard, Analytics, User Management, Targets.
- System Summary: Four stat cards — Total Targets (320), Completed (145, green), In Progress (98, yellow), and Pending (77, red).
- Target List Table: Sortable/filterable table showing Target ID, Customer Name, Address, Amount Due, assigned Officer, and Status (Completed / In Progress / Pending) with color-coded status labels.

Telegram Mini App Mock-up

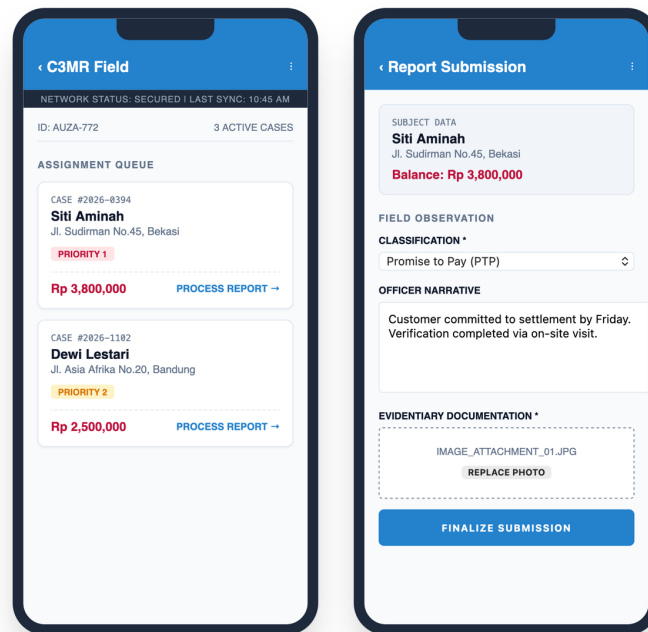


Figure A.3 Telegram Mini App Mock-up

The Telegram Mini App provides the following interface elements:

- Header: App name "C3MR Field App" with back navigation.
- Task Detail: Displays the current task with customer name, address, phone number, and amount due (highlighted in red).
- Payment Status Dropdown: Selectable options — Promise to Pay, Paid, Refused, Not Home, Partial Payment.
- Notes Field: Scrollable text area for the officer to enter visit notes.
- Upload Photo Evidence Button: Triggers the device camera or gallery.
- Submit Report Button: Sends the completed report to the backend API.

Manager Bot Mock-up

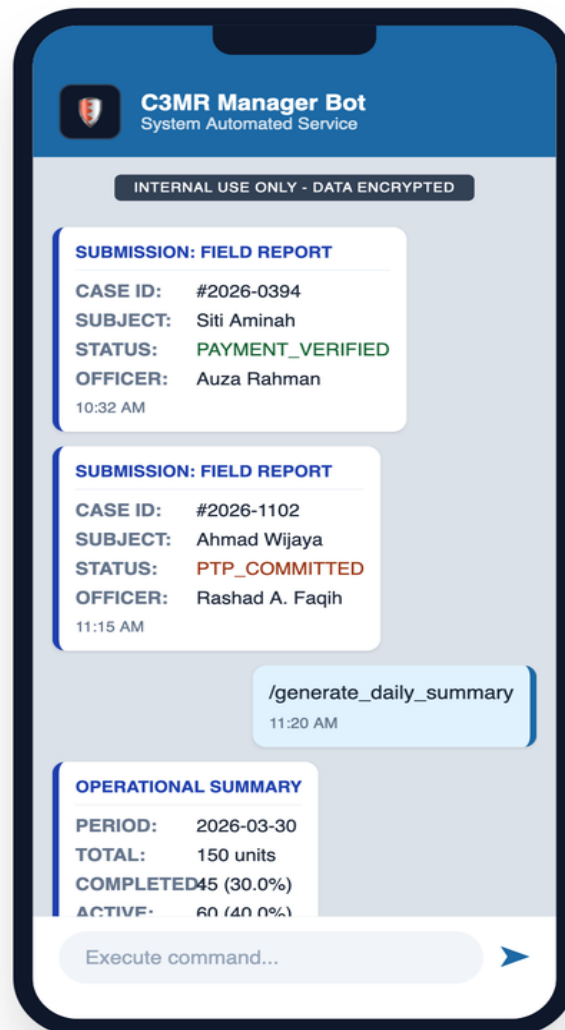


Figure A.4 Manager Bot Mock-up

The Manager Bot provides the following interface elements:

- Automatic Notifications: Each time a field officer submits a report, the bot sends a message to the manager's Telegram chat containing the customer name, payment status, officer name, and submission time.
- summary Command: The manager sends /summary and the bot replies with a real-time summary showing total targets, completed, in-progress, and pending counts.

B. HIERARCHICAL / ITERATIVE DESIGN

1. Data Flow Diagram

Level 0 – Context Diagram

The Level 0 (Context) Diagram shows the top-level data flows between the three external entities and the C3MR system:

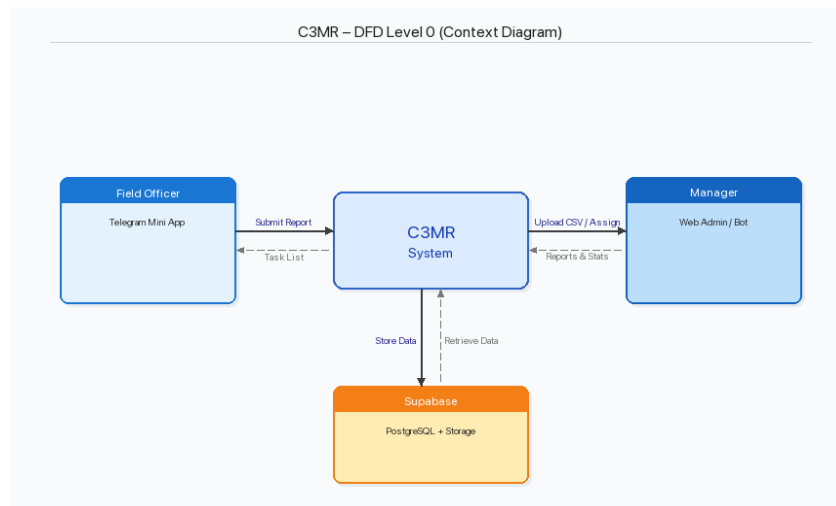


Figure B.0 DFD Level 0 – Context Diagram

- Level 1 – Major Processes

1.0 Authentication & Authorization: Handles user login (Supabase Auth for admin, Telegram user validation for officers), session management, and role-based access control.

2.0 Target Management: Manages CSV upload, parsing, validation, and assignment of collection targets to field officers.

3.0 Report Management: Processes field visit reports including form data validation, photo upload to object storage, and database insertion.

4.0 Officer Management: CRUD operations for field officer records, including linking Telegram user IDs.

5.0 Dashboard & Analytics: Aggregates data for the admin dashboard: total targets, completion rates, officer performance.

6.0 Notification Service: Sends task assignment notifications and status updates via the Telegram Bot API.

- **Level 2 – Report Submission & Notification Process (Detailed)**

The Level 2 diagram provides a detailed breakdown of the report submission and manager notification flow. This is illustrated in Figure B.3 – Sequence Diagram (see Section B.5 UML Diagrams), which shows the step-by-step interaction between the Field Officer, Telegram Mini App, FastAPI backend, Supabase database, Telegram Bot API, and Manager.

2. Interface Information Between Components

1. **Client ↔ Gateway:** Protocol: HTTPS (TLS 1.3); Data format: JSON; Authentication: Bearer token.
2. **Gateway ↔ Backend API:** Protocol: HTTP (internal); Headers forwarded: Authorization, X-Forwarded-For.
3. **Backend API ↔ Database:** Protocol: PostgreSQL wire (TCP); Interface: supabase-py SDK; Parameterized SQL queries.
4. **Backend API ↔ Object Storage:** Protocol: HTTPS; Interface: Supabase Storage API; Multipart file upload; Returns public URL.
5. **Backend API ↔ Telegram Bot API:** Protocol: HTTPS; HTTP POST with JSON; sendMessage / sendPhoto endpoints.

3. Software Engineering Design Steps

The project follows an Agile (Iterative) development methodology:

1. **Requirements Gathering:** Identify stakeholder needs, define user stories, and prioritize features in a product backlog.
2. **System Design:** Create architecture diagrams, define API contracts (OpenAPI spec), design database schema, and produce UI mock-ups.
3. **Sprint-based Implementation:** Develop features incrementally in 2-week sprints, starting with authentication, target upload, and report submission.
4. **Testing:** Unit tests (pytest), integration tests (API endpoint testing), and UAT with stakeholders.
5. **Deployment:** Deploy backend to cloud server, configure Supabase project, register Telegram bot, and publish Mini App.
6. **Review & Iteration:** Collect feedback, fix bugs, and iterate on features.

4. Component References and Libraries

1. Frontend – Web Admin Portal

- React.js (v18.x) – UI framework
- React Router (v6.x) – Client-side routing
- Axios – HTTP client for API calls
- Tailwind CSS – Utility-first CSS framework
- Papa Parse – CSV parsing library
- Chart.js / Recharts – Dashboard chart rendering

2. Frontend – Telegram Mini App

- Telegram Web App SDK – Native Telegram integration
- HTML5 / CSS3 / JavaScript – Core web technologies
- Fetch API – HTTP client for API calls

3. Backend

- Python (v3.11+) – Programming language
- FastAPI (v0.100+) – Web framework
- Pydantic (v2.x) – Data validation and serialization
- supabase-py – Supabase Python client SDK
- python-telegram-bot – Telegram Bot API wrapper
- Uvicorn – ASGI server
- python-multipart – File upload handling

4. Database & Storage

- Supabase – Backend-as-a-Service platform
- PostgreSQL (v15) – Relational database
- Supabase Storage – S3-compatible object storage
- Supabase Auth – Authentication service

5. UML Diagrams

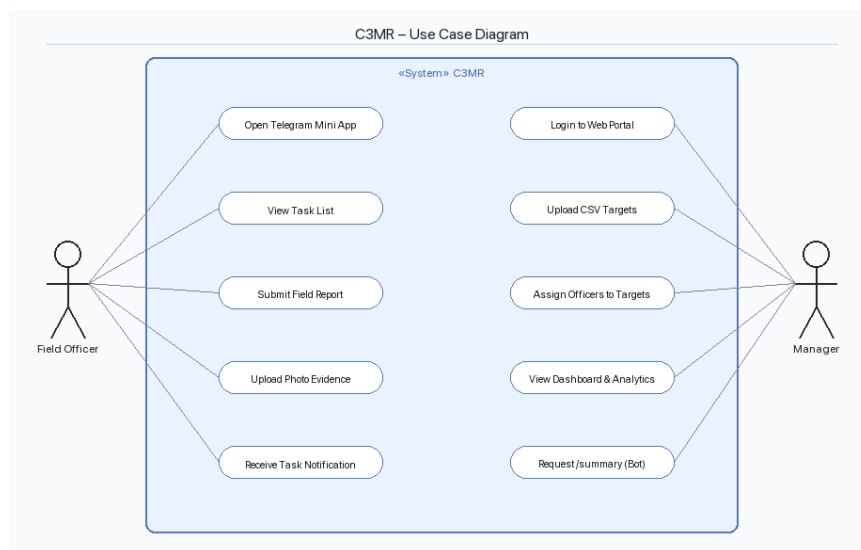


Figure B.1 Use Case Diagram

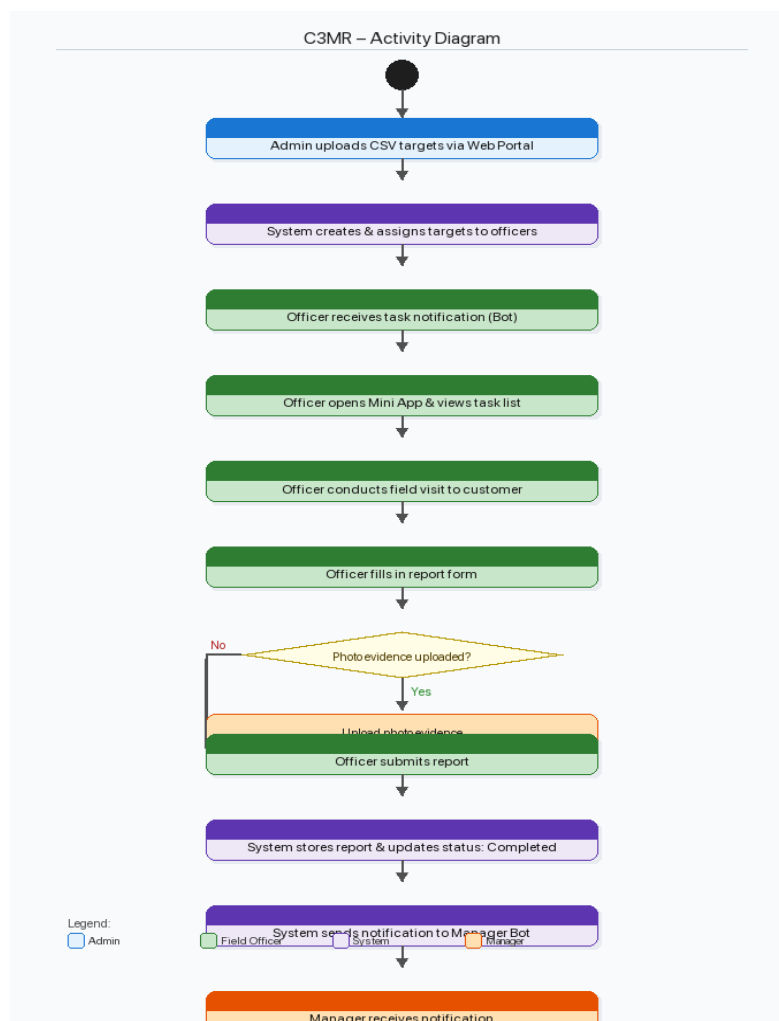


Figure B.2 Activity Diagram – C3MR Main Workflow

Figure B.3 Sequence Diagram – Report Submission & Manager Notification

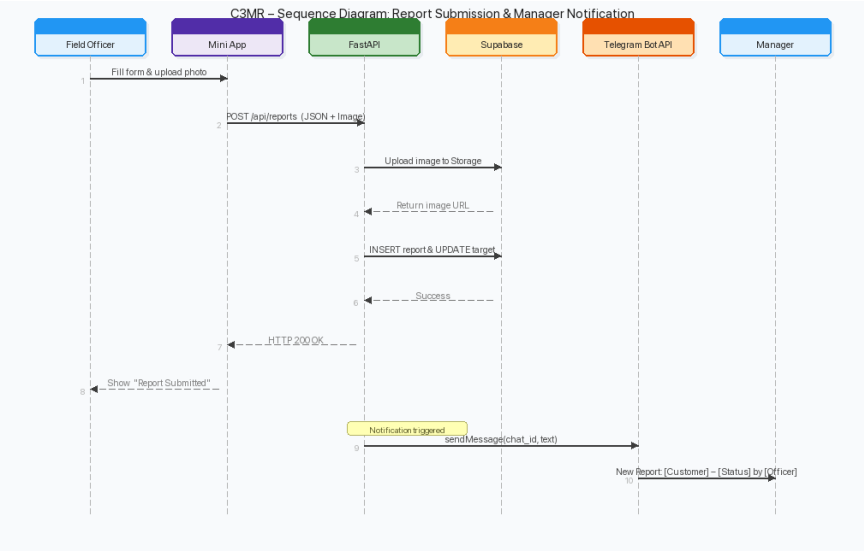


Figure B.4 Component Diagram

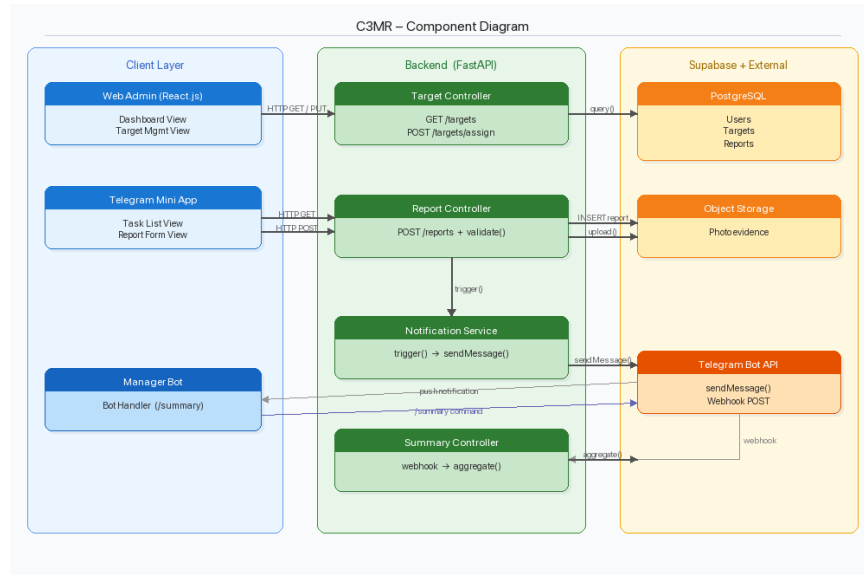
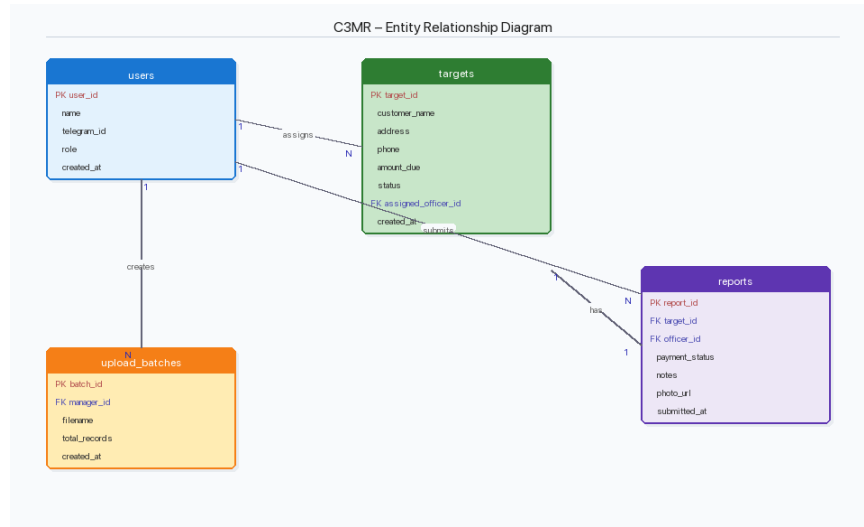


Figure B.5 Entity Relationship Diagram



C. STANDARDS USED

Category	Standard / Technology	Description
Data Format	JSON (RFC 8259)	All API request/response bodies use JSON.
API Architecture	REST (RESTful API)	Resource-based URLs, HTTP methods, status codes.
API Documentation	OpenAPI 3.0	FastAPI auto-generates OpenAPI 3.0 spec (Swagger UI).
Data Validation	Pydantic v2	Strict type checking and custom validators on all inputs.
Authentication	JWT (RFC 7519)	Supabase Auth issues JWTs upon login.
Encryption	TLS 1.3 (RFC 8446)	All client-server communication encrypted via HTTPS.
Database	SQL (PostgreSQL 15)	Standard SQL with PostgreSQL extensions.
Modeling	UML 2.5	Use case, class, sequence, activity, component, ERD.
Diagram Tool	Python Pillow (v10.x)	All UML & architecture diagrams rendered programmatically as PNG.
Code Formatter	Black (Python)	Line length 88 (default settings).
Code Formatter	Prettier (JS/TS)	Frontend JavaScript/TypeScript formatting.
Linter	Ruff (Python)	Fast, comprehensive style and error checking.
Linter	ESLint (JS/TS)	Frontend code linting with recommended rules.
CSS Framework	Tailwind CSS v3	Utility-first CSS for consistent, responsive styling.
Version Control	Git	GitHub as the remote repository host.
File Upload	Multipart/form-data (RFC 7578)	Photo evidence uploads.
CSV Format	RFC 4180	Target data CSV files.
Bot API	Telegram Bot API v7	Official Bot API for messages and Mini App integration.
Character Encoding	UTF-8	All text data throughout the system.
Date/Time Format	ISO 8601	All timestamps (e.g., 2024-01-15T10:30:00Z).

D. IMPLEMENTATION AND TESTING SCENARIO

A. Implementation Overview

The system is implemented in an iterative manner across four phases:

1. Phase 1 – Backend Foundation

- Set up FastAPI project with Uvicorn as the ASGI server.
- Configure Supabase project (PostgreSQL database, Storage buckets, Auth).
- Define database schema and create tables (users, targets, reports, upload_batches).
- Implement authentication middleware using Supabase JWT verification.
- Implement core API endpoints: user registration, target CRUD, report submission.

2. Phase 2 – Web Admin Portal

- Set up React.js project with Tailwind CSS.
- Implement login page with Supabase Auth integration.
- Build dashboard page with summary statistics and charts.
- Build target management page with CSV upload and officer assignment.
- Build report viewing page with filtering and search.

3. Phase 3 – Telegram Mini App & Bot

- Register Telegram bot via BotFather and configure webhook.
- Implement Mini App using Telegram Web App SDK.
- Build task list view and report submission form.
- Implement photo capture and upload functionality.
- Implement Manager Bot commands for notifications and quick actions.

4. Phase 4 – Integration & Testing

- End-to-end integration testing across all components.
- User Acceptance Testing (UAT) with stakeholders.
- Performance testing and optimization.
- Bug fixes and final adjustments.

B. Testing Scenarios

Scenario 1: Manager Login (Web Admin Portal)

Step	Action	Expected Result	Alternative
1	Manager navigates to the Web Admin Portal URL.	Login page is displayed with email and password fields.	If server is unreachable, show 'Unable to connect' error.
2	Manager enters valid email and password, clicks 'Login'.	System authenticates via Supabase Auth, redirects to Dashboard. JWT stored in local storage.	If credentials invalid, show 'Invalid email or password'. User stays on login page.
3	Manager enters email but leaves password empty.	Client-side validation shows 'Password is required'. No API call is made.	—
4	Manager's JWT token expires during session.	System detects 401 response, redirects to login with 'Session expired' message.	—

Scenario 2: Upload CSV Targets (Web Admin Portal)

Step	Action	Expected Result	Alternative
1	Manager clicks 'Choose CSV' button.	File picker opens, filtered to .csv files.	—
2	Manager selects valid CSV with correct columns.	File name displayed. Preview of first 5 rows shown.	If CSV has wrong columns, show format error.
3	Manager clicks 'Upload & Assign'.	System parses CSV, creates target records. Success: '150 targets uploaded.'	If any row has invalid data, show row-level error. Upload rejected.
4	Manager uploads CSV with 0 data rows.	Show 'CSV file contains no data rows' error.	—
5	Manager uploads a non-CSV file (e.g., .xlsx).	Client-side validation: 'Only .csv files are accepted.'	—

Scenario 3: Assign Targets to Officers (Web Admin Portal)

Step	Action	Expected Result	Alternative
1	Manager selects unassigned targets.	Selected targets highlighted. 'Assign' button becomes active.	—
2	Manager clicks 'Assign' and selects officer.	Targets assigned. Status → 'In Progress'. Telegram notification sent to officer.	If officer is at max capacity, show warning.
3	Manager attempts to reassign an already-assigned target.	Confirmation dialog appears. On confirm, reassignment proceeds.	If manager cancels, no changes made.

Scenario 4: Field Officer Views Task List (Telegram Mini App)

Step	Action	Expected Result	Alternative
1	Field Officer opens the Mini App from Telegram.	System validates Telegram user ID. Task list displayed.	If ID not registered, show 'Contact your manager' message.
2	Task list loads with assigned targets.	Each task shows customer name, address snippet, amount due, and status badge. Sorted by status (Pending first).	If no tasks, show 'No tasks assigned. Check back later.'
3	Officer taps on a task.	Task detail view opens with full customer info (name, address, phone, amount due).	–

Scenario 5: Field Officer Submits Report (Telegram Mini App)

Step	Action	Expected Result	Alternative
1	Officer opens task detail and taps 'Submit Report'.	Report form displayed with Payment Status dropdown, Notes, and Upload button.	–
2	Officer selects payment status (Promise to Pay / Paid / Refused / Not Home / Partial Payment).	Selection recorded. Dropdown shows selected value.	–
3	Officer types notes.	Text recorded in notes field.	–
4	Officer taps 'Upload Photo Evidence'.	Camera or gallery opens.	If permission denied, show 'Please grant camera access.'
5	Officer captures/selects a photo.	Photo thumbnail preview displayed.	If photo > 10 MB, show size limit error.
6	Officer taps 'Submit Report' with all fields filled.	POST /api/reports sent. Photo uploaded. Report inserted. Target → 'Completed'. Success message shown. Officer returned to task list.	If network error, show retry message. Form data preserved.
7	Officer taps 'Submit Report' without payment status.	Validation error: 'Please select a payment status.' Form not submitted.	–
8	Officer taps 'Submit Report' without photo.	Validation error: 'Please upload photo evidence.' Form not submitted.	–

Scenario 6: Manager Views Dashboard (Web Admin Portal)

Step	Action	Expected Result	Alternative
------	--------	-----------------	-------------

1	Manager navigates to Dashboard page.	Dashboard loads with 4 summary cards (Total Targets, Completed, In Progress, Pending) and completion rate chart.	If no data, show 'Upload targets to get started.'
2	Manager views Target List table.	Table shows: ID, Customer, Address, Amount Due, Officer, Status, Actions. Supports sorting and filtering by status.	—
3	Manager clicks on a completed target row.	Target detail view shows customer info and submitted report (payment status, notes, photo evidence).	—

Scenario 7: Manager Bot Notifications (Telegram)

Step	Action	Expected Result	Alternative
1	A field officer submits a report.	Manager receives Telegram notification: '  New Report: [Customer] – [Payment Status] by [Officer].'	If bot blocked by manager, notification fails silently. Log error.
2	Manager sends /summary to the bot.	Bot responds with total targets, completed, pending count, and completion %.	If no targets, bot replies 'No targets found.'

REFERENCES

- Bray, T. (Ed.). (2017). The JavaScript Object Notation (JSON) data interchange format (RFC 8259). Internet Engineering Task Force (IETF).
<https://datatracker.ietf.org/doc/html/rfc8259>
- Jones, M., Bradley, J., & Sakimura, N. (2015). JSON web token (JWT) (RFC 7519). Internet Engineering Task Force (IETF).
<https://datatracker.ietf.org/doc/html/rfc7519>
- Meta Open Source. (n.d.). React documentation. Meta. <https://react.dev/>
- Pillow Contributors. (n.d.). Pillow (PIL fork) documentation.
<https://pillow.readthedocs.io/>
- Pydantic. (n.d.). Pydantic documentation. <https://docs.pydantic.dev/>
- Ramírez, S. (n.d.). FastAPI documentation. Tiangolo. <https://fastapi.tiangolo.com/>
- Supabase. (n.d.). Supabase documentation. <https://supabase.com/docs>
- Tailwind Labs. (n.d.). Tailwind CSS documentation. <https://tailwindcss.com/docs>
- Telegram. (n.d.). Telegram bot API documentation. <https://core.telegram.org/bots/api>
- Telegram. (n.d.). Telegram mini apps documentation.
<https://core.telegram.org/bots/webapps>